EXP5: Missionaries and Cannibals Problem

Name: Y. Roshitha (192424400)

Aim:

To develop a Python program that solves the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm, finding a safe sequence of boat crossings that transports all missionaries and cannibals from the left bank to the right bank without the cannibals ever outnumbering the missionaries on either bank.

Algorithm:

Step 1: Start with the initial state: 3 missionaries and 3 cannibals on the left bank, boat on the left.
Step 2: Check if the current state is the goal state (all missionaries and cannibals on the right bank).
Step 3: Generate all valid next states by moving the boat with one of the following combinations:

- 1 missionary
- 2 missionaries
- 1 cannibal
- 2 cannibals
- 1 missionary and 1 cannibal

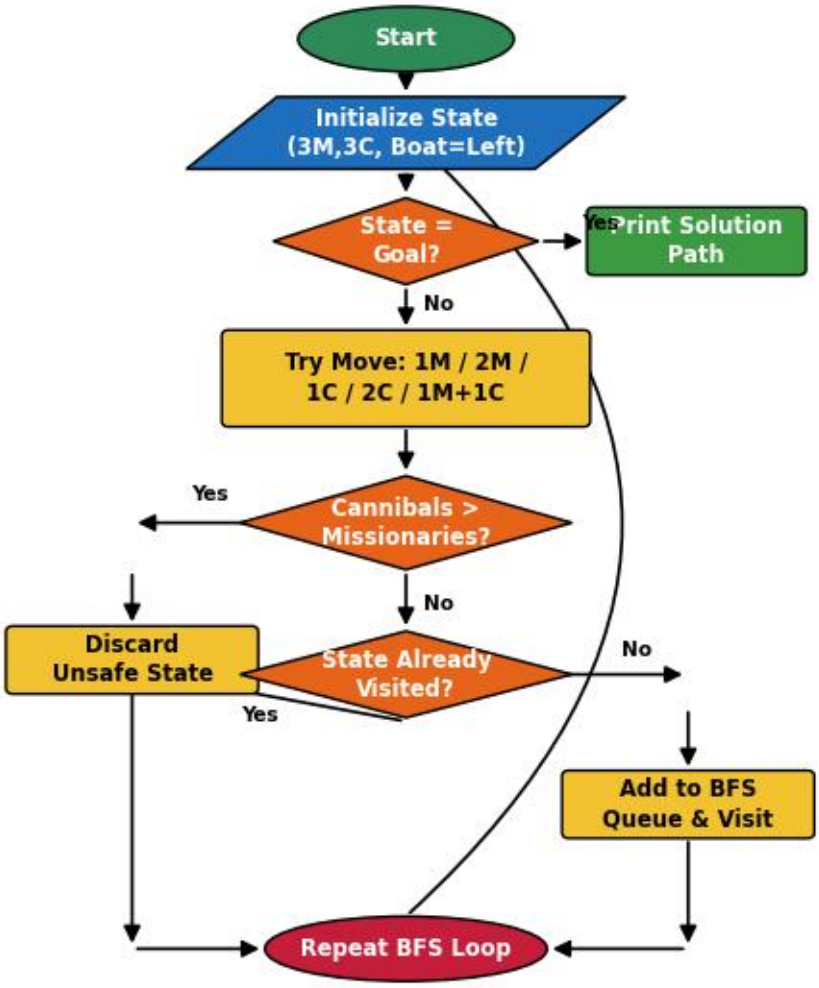
Step 4: Discard any state where cannibals outnumber missionaries on either bank (unsafe state).
Step 5: Add all valid, unvisited states to the BFS queue.
Step 6: Mark the current state as visited.
Step 7: Repeat Steps 2-6 until the goal state is found.
Step 8: Return the complete path from the initial state to the goal state.

Source Code:

```
from collections import deque

def is_valid(state):
    m_left, c_left, boat, m_right, c_right = state
    if m_left < 0 or c_left < 0 or m_right < 0 or c_right < 0:
        return False
    if m_left > 0 and m_left < c_left:
        return False
    if m_right > 0 and m_right < c_right:
        return False
    return True

def get_next_states(state):
    m_left, c_left, boat, m_right, c_right = state
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
    next_states = []

    for m, c in moves:
        if boat == 1:  # boat on left, move to right
            new_state = (m_left - m, c_left - c, 0, m_right + m, c_right + c)
        else:  # boat on right, move to left
```

```
            new_state = (m_left + m, c_left + c, 1, m_right - m, c_right - c)

        if is_valid(new_state):
            next_states.append(new_state)

    return next_states

def bfs(start, goal):
    visited = set([start])
    queue = deque([(start, [start])])

    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path

        for next_state in get_next_states(state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))

    return None

start = (3, 3, 1, 0, 0)   # 3 missionaries, 3 cannibals, boat on left
goal = (0, 0, 0, 3, 3)    # all on right

solution = bfs(start, goal)

print("Steps to solve the Missionaries and Cannibals Problem:\n")
if solution:
    step = 1
    for state in solution:
        m_left, c_left, boat, m_right, c_right = state
        side = "Left" if boat == 1 else "Right"
        print(f"Step {step}: Left(M={m_left}, C={c_left})  "
              f"Right(M={m_right}, C={c_right})  Boat: {side}")
        step += 1
else:
    print("No solution found.")
```

Flowchart:


Start
Initialize State
(3M,3C, Boat=Left)
State =
Goal?
Yes
Print Solution
Path
No
Try Move: 1M / 2M /
1C / 2C / 1M+1C
Yes
Cannibals >
Missionaries?
No
Discard
Unsafe State
State Already
Visited?
No
Yes
Add to BFS
Queue & Visit
Repeat BFS Loop

Output:

```
=============== RESTART: C:/Users/yanar/Documents/CSA1717/EXP5.py ==============
Steps to solve the Missionaries and Cannibals Problem:

Step 1: Left(M=3, C=3)  Right(M=0, C=0)  Boat: Left
Step 2: Left(M=3, C=1)  Right(M=0, C=2)  Boat: Right
Step 3: Left(M=3, C=2)  Right(M=0, C=1)  Boat: Left
Step 4: Left(M=3, C=0)  Right(M=0, C=3)  Boat: Right
Step 5: Left(M=3, C=1)  Right(M=0, C=2)  Boat: Left
Step 6: Left(M=1, C=1)  Right(M=2, C=2)  Boat: Right
Step 7: Left(M=2, C=2)  Right(M=1, C=1)  Boat: Left
Step 8: Left(M=0, C=2)  Right(M=3, C=1)  Boat: Right
Step 9: Left(M=0, C=3)  Right(M=3, C=0)  Boat: Left
Step 10: Left(M=0, C=1)  Right(M=3, C=2)  Boat: Right
Step 11: Left(M=1, C=1)  Right(M=2, C=2)  Boat: Left
Step 12: Left(M=0, C=0)  Right(M=3, C=3)  Boat: Right
>>>
```

Result:

The Missionaries and Cannibals program successfully found a safe sequence of boat crossings that transported all 3 missionaries and 3 cannibals from the left bank to the right bank. The Breadth-First Search algorithm systematically explored all valid boat moves and discarded any state where the cannibals outnumbered the missionaries on either bank.

During execution, the program displayed every intermediate state, showing the number of missionaries and cannibals on each bank along with the position of the boat, until the goal state was reached in 12 steps.

Thus, the Missionaries and Cannibals Problem was solved successfully using the Breadth-First Search (BFS) algorithm.